

```
Hackintosh:~ # ip addr
1: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 65526 qdisc mq master state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state DOWN group default qlen 1000
    link/ether 28:02:04:01:70:ce brd ff:ff:ff:ff:ff:ff
```

Figure 10: MTU value shown by *ip addr* command

**Caution:** If your system generates high traffic as in the case of a hosting Web server, you must not disable it.

## Pause frames

A pause frame is the duration of the pause, in milliseconds, which the adapter issues to the network switch to stop sending packets if the ring buffer gets full. If the pause frame is enabled, then loss of packets will be minimal.

To view the pause frame setting, use the following command:

```
$ethtool -a eth0
```

To enable pause frames for both RX (receiving) and TX (transferring), run the command given below:

```
$ethtool -A eth0 rx on tx on
```

There are lots of other things you can tune using *ethtool* to increase performance through the network adapter, like speed and duplex. But these are beyond the scope of this article. You can check them on the *ethtool* manual.

## Software interrupts

**SoftIRQ timeout:** If the software interrupt doesn't process packets for a long time, it may cause the NIC buffer to overflow and, hence, can cause packet loss. *netdev\_budget* shows the default value of the time period for which *softirq* should run.

```
$sysctl net.core.netdev_budget
```

The default value should be 300 and may have to be increased to 600 if you have a high network load or a 10Gbps (and above) system.

```
# sysctl -w net.core.netdev_budget=600
```

**IRQ balance:** Interrupt requests are handled by different CPUs in your system in a round robin manner. Due to regular context switching for interrupts, the request handling timeout may increase. The *irqbalance* utility is used to balance these interrupts across the CPU and has to be turned off for better performance.

```
$service irqbalance stop
```

If your system doesn't deal with high data receiving and

```
Hackintosh:~ # sysctl net.ipv4.tcp_rmem
net.ipv4.tcp_rmem = 4096      87380  6291456
Hackintosh:~ # sysctl net.ipv4.tcp_wmem
net.ipv4.tcp_wmem = 4096      16384   4194304
```

Figure 11: TCP read and send buffer size

transmission (as in the case of a home user), this configuration will be helpful to increase performance. But for industrial use, it has to be tweaked for better performance, as stopping it may cause high load on a single CPU – particularly for services that cause fast interrupts like *ssh*. You can also disable *irqbalance* for particular CPUs. I recommend that you do that by using */etc/sysconfig/irqbalance*. To see how to balance specific CPUs, visit <http://honglus.blogspot.in/2011/03/tune-interrupt-and-process-cpu-affinity.html>.

## Kernel buffer

The socket buffer size is:

```
net.core.wmem_default
net.core.rmem_default
net.core.rmem_max
net.core.wmem_max
```

These parameters show the default and maximum write (receiving) and read (sending) buffer size allocated to any type of connection. The default value set is always a little low since the allocated space is taken from the RAM. Increasing this may improve the performance for systems running servers like NFS. Increasing them to 256k/4MB will work best, or else you have to benchmark these values to find the ideal value for your system's configuration.

```
$sysctl -w net.core.wmem_default=262144.
$sysctl -w net.core.wmem_max=4194304
$sysctl -w net.core.rmem_default=262144
$sysctl -w net.core.rmem_max=4194304
```

Every system has different values and increasing the default value may improve its performance but you have to benchmark for every value change.

## Maximum queue size

Before processing the data by the TCP/UDP layer, your system puts the data in the kernel queue. The *net.core.netdev\_max\_backlog* value specifies the maximum number of packets to put in the queue before delivery to the upper layer. The default value is not enough for a high network load, so simply increasing this value cures the performance drain due to the kernel. To see the default value, use *sysctl* with *\$sysctl net.core.netdev\_max\_backlog*. The default value is 1000 and increasing it to 3000 will be enough to stop packets from being dropped in a 10Gbps (or more) network.